Basic (your first coroutine) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, plus how this actually bites you in a real program). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

You need a function that waits a second, then hands back a value — but written the `async` way instead of the normal way.

Here's the part that trips everyone up the first time: writing `async def` in front of a function does **not** make it run like a normal function. Calling it, like `wait_and_return()`, doesn't run the code inside at all. It just hands you back a "coroutine object" — a kind of promise that the code will run later, if someone actually starts it.

Things to use:

- `async def wait_and_return():` — defines the coroutine function.
- `await asyncio.sleep(1)` — pauses without blocking anything else.
- `asyncio.run(...)` — actually starts and runs a coroutine from normal, top-level code.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Same idea, with the exact vocabulary. Writing `async def wait_and_return() -> str:` defines a **coroutine function**. Calling `wait_and_return()` does not execute its body — it constructs and returns a **coroutine object**, a suspended unit of work that hasn't started yet. Python will even print a `RuntimeWarning: coroutine 'wait_and_return' was never awaited` if you build one and never do anything else with it — that warning exists specifically to catch this exact mistake.

The exact pieces:

- `async def wait_and_return() -> str:` — the `async` keyword marks this as a coroutine function; the `-> str` documents what it eventually resolves to.
- `await asyncio.sleep(1)` — the async equivalent of `time.sleep`; hands control back to the event loop for that second instead of blocking anything.
- `return "done"` — an ordinary `return`. It sets what the coroutine resolves to once it finishes; it does not run the function immediately.
- `asyncio.run(wait_and_return())` — `wait_and_return()` builds the coroutine object; `asyncio.run(...)` is what actually starts the event loop, drives it to completion, and hands back the return value.

Think about where this code actually gets called from in a real program, not just a standalone script. `asyncio.run(...)` assumes it's the *only* thing starting an event loop — it creates one, runs your coroutine, and tears the loop down afterward. That assumption breaks the moment this code runs somewhere a loop is already running: inside a FastAPI route (Doc12), inside a Jupyter notebook, inside any other `async def` function. Call `asyncio.run(...)` from any of those places and you get `RuntimeError: asyncio.run() cannot be called from a running event loop` — not a bug in your coroutine, a bug in *where* you tried to start it.

The real design question isn't just "how do I run this coroutine" — it's "does the code that calls this function control its own event loop, or does it need to work whether or not one is already running?"

The extra piece that answers that question:

- **Don't call `asyncio.run()` inside reusable functions.** Keep it at the true entry point of your program (the `if __name__ == "__main__":` block) and nowhere else. Any function meant to be reused from inside an already-async caller should just be `async def` and get `await`ed directly by whatever calls it — never wrapped in its own `asyncio.run()`.
- `asyncio.get_running_loop()` (inside a `try/except RuntimeError`) is how you can detect, at runtime, whether a loop is already running — useful in library code that genuinely needs to

support being called both ways, though the cleaner fix is almost always to just not call `asyncio.run()` from library code at all.

Difference between Basic and Intermediate: Basic names the plain idea and the 3 tools you need for the tidy, single-script case. Intermediate gives the same idea the exact vocabulary, shows precisely what each piece does, and asks where this code will actually be called *from* in a real project — a standalone script is the only place `asyncio.run()` is ever safe to call, and the same "coroutine that just needs `await`" you write here will need to plug directly into someone else's already-running loop the moment it's reused, which is exactly what the next exercise (`gather_speed`) and the `async_client_conversion` exercise both build toward.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make an async function called wait_and_return that takes nothing:
    wait 1 second (without blocking anything else)
    give back the value "done"

first, show the mistake:
    call wait_and_return() and print it directly, with no await, no asyncio.run
    -> this prints a coroutine object, not "done"

now, do it correctly:
    use asyncio.run(...) to actually start and run wait_and_return()
    store what it gives back
    print that -> this prints "done"
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# coroutine_basics_practice.py
import asyncio

async def wait_and_return():
    await asyncio.sleep(1)
    return "done"

result = asyncio.run(wait_and_return())
```

Expected output if you run just this: nothing yet — add `print(result)` below it to see `done` print. Add a line above the `asyncio.run(...)` call that prints `wait_and_return()` directly (no `await`, no `asyncio.run`) to see the coroutine-object mistake for yourself.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
import asyncio

define:
    async def wait_and_return() -> str:
        await asyncio.sleep(1)
        return "done"

demonstrate the gotcha:
    unstarted = wait_and_return()
    print(unstarted)                    # coroutine object, not "done"

run it for real:
    result: str = asyncio.run(wait_and_return())
    print(result)                       # "done"
```

```
# coroutine_basics_practice.py
async def wait_and_return() -> str:
    await asyncio.sleep(1)
    return "done"


if __name__ == "__main__":
    result: str = asyncio.run(wait_and_return())
```

Fill in the `unstarted = wait_and_return()` demonstration and both `print(...)` calls yourself, then compare against the Solution.

Once that's working, add a timeout, the way a real caller should — a coroutine that never actually finishes (a network call that hangs, say) shouldn't be able to freeze your whole program forever:

```
import asyncio

define wait_and_return, same as before

try:
    result = asyncio.run(asyncio.wait_for(wait_and_return(), timeout=5))
    print(result)
```

```
except asyncio.TimeoutError:
    print("wait_and_return took too long, giving up")
```

```
# coroutine_basics_practice.py
import asyncio


async def wait_and_return() -> str:
    await asyncio.sleep(1)
    return "done"


async def main() -> None:
    try:
        result = await asyncio.wait_for(wait_and_return(), timeout=5)
        print(result)
    except asyncio.TimeoutError:
        print("wait_and_return took too long, giving up")


if __name__ == "__main__":
    asyncio.run(main())
```

Notice `asyncio.run()` now only appears once, at the very bottom, wrapping `main()` — exactly the Hint 1 point about keeping it at the true entry point. Try lowering `timeout=5` to `timeout=0.5` yourself (shorter than the 1-second sleep) to see the `except` branch actually trigger, then compare your finished versions against the Solution.

One more tool: `asyncio.create_task(coro)` starts a coroutine running in the background straight away. Keep the task in a variable, do other awaited work, then `await task` to collect its result.

Difference between Basic and Intermediate: Basic just runs `wait_and_return()` and trusts it to finish. Intermediate wraps the same coroutine in `asyncio.wait_for(..., timeout=...)`, which is what a real caller should almost always do around anything that waits on the outside world — without it, one hung network call can freeze your whole program indefinitely, with no way to recover short of killing the process.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution